Lab 10: Path Planning on a Grid

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1 Overview

In Lab 9 you built the **perception** half of the sense–plan–act loop: a log-odds occupancy grid with a binary `planner_blocked` mask as its final output. This week you connect that mask to the **plan** half.

You will:

1. Grow obstacles by the robot's radius — the Minkowski-sum "point-robot trick" — so the planner can pretend the robot is a single cell.
2. Implement **Dijkstra's algorithm** on the 8-connected grid and watch it fan out equally in all directions.
3. Change one line (priority $= g + h$) to get **A*** with the octile heuristic, and see how many fewer cells it expands for the same optimal path.
4. **Smooth** the grid-aligned path into a short, waypoint-based path using line-of-sight shortcutting.

By the end you have a planning stack: `occupancy grid` → `C-space` → `path of cells` → `sparse waypoints` — exactly the chain that feeds into Week 9's trajectory controller.

Duration: 2 hours

Software required:

- Python environment from Lab 0
- Libraries: `numpy`, `matplotlib`, `scipy`, `scikit-image`

2 Learning Objectives

By the end of this lab, you will be able to:

1. Compute a C-space mask from a binary occupancy grid by inflating obstacles with a distance transform.
2. Implement Dijkstra's shortest-path search on an 8-connected grid with cardinal cost 1 and diagonal cost $\sqrt{2}$.
3. Extend Dijkstra to A* by adding the octile heuristic and explain why the result is still optimal.
4. Compare Dijkstra and A* by counting nodes expanded — visualize the "fan" vs. the "beam".
5. Smooth a grid-aligned path into sparse waypoints using line-of-sight shortcutting.

3 Pre-Lab: Quick Review

Complete this section **before** coming to lab.

3.1 C-space inflation

If the robot has radius r and the grid resolution is res, how many grid cells should each obstacle grow by?

$$\text{inflation (cells)} = \lceil r/res \rceil.$$

For a robot of radius 0.20 m on a 0.10 m grid, that is **2 cells** of inflation.

3.2 Octile heuristic

For two cells $a = (i_a, j_a)$ and $b = (i_b, j_b)$, let $\Delta i = |i_a - i_b|$ and $\Delta j = |j_a - j_b|$. On an 8-connected grid with cardinal cost 1 and diagonal cost $\sqrt{2}$, the shortest path length in the absence of obstacles is the **octile** distance:

$$h(a, b) = (\Delta i + \Delta j) + (\sqrt{2} - 2) \cdot \min(\Delta i, \Delta j).$$

Sanity check: this is $\min(\Delta i, \Delta j)$ diagonal moves plus $|\Delta i - \Delta j|$ cardinal moves.

3.3 Pre-lab questions

Q1. A robot of radius 0.25 m plans on a grid with resolution 0.05 m. What is the inflation radius in cells?

Q2. From $(2, 3)$ to $(9, 7)$ on an 8-connected grid, what is the octile heuristic value?

Q3. A* with an **admissible** heuristic ($h(n) \leq$ true cost to goal) is guaranteed to return an optimal path. Is octile admissible on an 8-connected grid? One sentence.

Answers are at the bottom of the notebook.

4 Part 1: C-space Inflation (30 min)

4.1 Step 1.1 — Load or build an occupancy map

The notebook provides a helper `make_test_map()` that returns a binary 100×100 grid with walls, an L-shaped obstacle, and one internal divider. You can replace this with the `planner_blocked` mask from Lab 9 if you want.

4.2 Step 1.2 — Inflate with a distance transform

Use `scipy.ndimage.distance_transform_edt` to compute, for every free cell, the distance (in cells) to the nearest obstacle. Then threshold:

```
free = (grid == 0)
dist = distance_transform_edt(free)
inflated = (dist <= r_cells).astype(np.uint8)
```

Visualize the original vs. inflated map side by side.

4.3 Step 1.3 — Discussion

Does it matter that the inflation uses **Euclidean** distance instead of Manhattan? When would a rectangular robot need something other than a disk inflation? One line each.

5 Part 2: Dijkstra's Algorithm (30 min)

5.1 Step 2.1 — 8-connected neighbors

Given cell (i, j) in an $M \times N$ grid, yield every in-bounds neighbor with step cost 1 (cardinal) or $\sqrt{2}$ (diagonal). Write `neighbors_8(i, j, M, N)` as a generator.

5.2 Step 2.2 — Dijkstra

Implement `dijkstra(grid, start, goal)` using `heapq`. Keep a `g` array of best-known distances and a `parent` dict for path reconstruction. Return:

- the path as a list of `(i, j)` cells,
- its cost,
- the `g` array (for visualization),
- a `closed_order` array storing *when* each cell was first popped from the queue.

5.3 Step 2.3 — Visualize

Plot the inflated map with:

- the path in red,
- cells **expanded** (popped) by the search, colored by expansion order.

Run it from start $(5, 5)$ to goal $(95, 95)$ on the test map.

6 Part 3: A* with the Octile Heuristic (30 min)

6.1 Step 3.1 — Heuristic

Implement `octile(a, b)` with the formula from the pre-lab.

6.2 Step 3.2 — A*

Copy your Dijkstra, then change **one line**: push cells onto the queue with priority $g + h$ instead of just g. Keep both `g` and `f` for cleanliness.

6.3 Step 3.3 — Compare

Run both algorithms on the same start/goal pair. Compare:

- Path cost: should be **identical**.
- Number of cells expanded: A* should expand far fewer.
- Visually, Dijkstra makes a circular fan; A* makes a beam aimed at the goal.

6.4 Step 3.4 — Short answer

The octile heuristic is admissible on an 8-connected grid with unit cardinals and diagonal cost $\sqrt{2}$. In one sentence, why?

7 Part 4: Path Smoothing (30 min)

7.1 Step 4.1 — Line of sight

Write `has_line_of_sight(grid, a, b)` that returns `True` when every cell on the Bresenham line from a to b is free. Use `skimage.draw.line`.

7.2 Step 4.2 — Greedy shortcut

Walk the grid path and, at each step, skip ahead to the **farthest** cell that still has line-of-sight from the current cell. The result is a sparse list of waypoints the robot can steer toward.

7.3 Step 4.3 — Visualize

Plot the raw grid path, the shortcut waypoints, and the straight lines between them on the same axes. Count the number of waypoints before and after.

7.4 Step 4.4 — Short answer

On an 8-connected grid the raw path zigzags even when a straight line is clear. Why? What information does the grid search throw away that a line-of-sight shortcut recovers? Two sentences.

8 Wrap-up

Your final output — a short list of waypoints in world coordinates — is the input to a trajectory controller (Week 9). That closes the sense–plan–act loop:

1. **Sense**: ray-cast LiDAR into an occupancy grid (Lab 9, Part 2).
2. **Interpret**: threshold to a binary map and inflate by robot radius (this lab, Part 1).
3. **Plan**: search the grid with A* for a path of cells (this lab, Parts 2–3).
4. **Smooth**: collapse the path to sparse waypoints with line-of-sight (this lab, Part 4).
5. **Act**: drive a trajectory between waypoints (Week 9).